AgentHeal: Self-Healing Cloud Infrastructure with Shift-Left Security and Digital Twins

Aadi Haldar, Raghuram Sekar, Aaditya Paul, Shravan Rajesh Menon
Department of Artificial Intelligence & Engineering
Amrita Vishwa Vidyapeetham, Coimbatore, India

***Abstract*—Production Kubernetes systems face two problems that constantly get in each other's way. Security flaws slip past code review because linters cannot see runtime behavior. At the same time, autoscalers react blindly to CPU spikes without knowing what caused them, which often turns minor glitches into full outages. We designed AgentHeal to connect both worlds. Before code merges, our GitHub App parses pull requests using AST checks for OWASP Top 10 vulnerabilities, flags unpatched dependencies, and blocks merges if the compliance score drops below 60. When a PR fails, an agent powered by Gemini 2.0 Flash writes a patch and opens a fix branch automatically. At runtime, our digital twin samples cluster metrics every five seconds. An in-memory Isolation Forest flags anomalies in 2.26 ms, KernelSHAP points out the offending metric, and two agents (a SimiFed Q-learning agent and an LLM agent) propose recovery actions in parallel. Before touching the cluster, we test the chosen action in an $M/M/c$ queue simulation built with SimPy. If the simulation predicts queue saturation ($\rho \geq 0.95$ or post-remediation utilization over 80%), the action is canceled. In benchmarks on Google Online Boutique across 60 injected faults, detection took 2.26 ms and recovery took 2.98 s on average. That cuts recovery time by over 98% compared to standard autoscalers. On 50 synthetic pull requests, our pre-merge scanner reached 98.5% precision, and in a live test on GitHub, it detected and fixed 14 OWASP vulnerabilities without human help.**

***Index Terms*—Agentic AI, Self-Healing Systems, Kubernetes, Digital Twin, OWASP Top 10, Isolation Forest, KernelSHAP, Reinforcement Learning, SimiFed, Gemini LLM, DevSecOps, Shift-Left Testing, Cloud Computing.**

I. Introduction

Microservice applications running on Kubernetes are notoriously hard to keep reliable. Two issues cause most of the headaches. First, developers inadvertently merge code that carries security flaws or performance bugs. In fact, DORA's 2024 report attributes more than 60% of unplanned downtime to preventable code and configuration mistakes [1]. Second, when things break in production, standard tools handle it poorly. An on-call engineer might take 30 to 45 minutes to notice and fix a problem. Meanwhile, the Kubernetes Horizontal Pod Autoscaler (HPA) only watches aggregate CPU and memory. If a pod locks up in a deadlock or leaks memory, spinning up more replicas just burns cluster resources without fixing anything [1], [9].

Traditionally, software teams treat these as separate jobs. Shift-Left tools like Bandit [6] or SonarQube [7] look for bugs during CI, but they have zero visibility into runtime traffic and cannot generate tested fixes. Shift-Right tools like Prometheus and chaos engineering suites [24], [23] alert operators after things break, but they lack root-cause analysis and offer no pre-actuation safety checks. Recent projects have explored digital twins [12] and LLM-assisted repair [15], but nobody has tied the two into a single closed loop with safety validation.

We built AgentHeal to bridge this divide. It combines pre-merge security review with post-deployment self-healing over a shared event bus. Before merge, a GitHub App inspects code changes against the OWASP Top 10:2021 standards, scans dependencies for CVEs, and blocks merges until issues are resolved. A Gemini 2.0 Flash agent writes and submits fix pull requests automatically. In production, our digital twin streams metrics, flags anomalies in 2.26 ms using an Isolation Forest, attributes root causes with KernelSHAP, and runs a SimiFed RL agent alongside an LLM. Crucially, before executing any control-plane command, an $M/M/c$ queue simulation in SimPy validates that the fix will not crash downstream services.

We make three main contributions:

1) *A Closed-Loop Architecture*: We link pre-merge pull-request auditing directly with runtime Kubernetes recovery using a shared event bus and cluster topology graph.
2) *A Safety-Gated Remediation Path*: We combine unsupervised anomaly detection, KernelSHAP attribution, dual-agent consensus, and a real-time SimPy $M/M/c$ simulation gate that aborts destabilizing actions before they hit the cluster.
3) *Empirical Evaluation*: We test AgentHeal on Google Online Boutique across four fault scenarios ($N = 15$ trials each), report on a 50-PR security benchmark, and walk through a live, end-to-end pull request remediation on GitHub.

II. Related Work

Table I compares existing approaches across cloud reliability and DevSecOps compliance.

Early automated remediation relied mainly on static runbooks. Keptn [9] and Argo Rollouts [10] automate deployment rollbacks and canaries, but only follow rules written by engineers beforehand. If an unexpected failure crops up, the runbook cannot help. This limitation is well documented in SRE practice [23] and explains why chaos engineering [24] is used to check known failure modes rather than resolve novel ones.

On the machine learning side, multi-agent reinforcement learning works well for dynamic cloud provisioning [11], and the convergence of Q-learning [27] is thoroughly proven. FedAvg [28] showed how distributed models can train locally without exposing raw data. SF-DTM [12] brought this into cloud management by matching cluster states with baseline vectors in a digital twin. We use SF-DTM as our foundation and add an LLM reasoning partner, KernelSHAP attribution, and an on-path SimPy simulation gate.

For anomaly detection, Isolation Forest [13] and LOF [29] are common in AIOps. Commercial platforms often report 50–200 ms detection times [14]; we reach 2.26 ms by caching our trained model in memory. We prefer KernelSHAP [16] over LIME [30] because Shapley values provide mathematical guarantees of efficiency, symmetry, and additivity, keeping explanations consistent across trials.

In automated program repair, AutoCodeRover [15] showed that LLMs can find and patch real bugs, though it works outside CI/CD pipelines. AgentHeal uses Gemini 2.0 Flash [17] in a ReAct loop [18] to draft pull requests that must pass unit tests before submission. For security scoring, we target OWASP Top 10:2021 [4], with plans to support OWASP Top 10:2025 [31]. We currently scan dependencies against a curated advisory list rather than live feeds like NVD [32], OSV [33], or Dependabot [34]. Finally, discrete-event simulation has a long history in DEVS [20] and queueing theory [19]; our twist is running SimPy as a fast sub-12 ms gate right before executing cluster commands.

III. Methodology

A. End-to-End Self-Healing Architecture Overview

Fig. 1 sketches how AgentHeal operates. Two autonomous loops run in tandem across a shared event bus. On the pre-merge side, the Shift-Left engine intercepts code changes, checks the AST against OWASP rules, verifies dependencies, and uses Gemini 2.0 Flash to submit fixes if tests fail. On the operations side, the Shift-Right engine continuously watches cluster telemetry. Every five seconds, Prometheus pulls metrics, the digital twin updates, and an Isolation Forest checks for anomalies. If an alert fires, KernelSHAP isolates the primary metric and hands it to both a SimiFed RL agent and an LLM agent. If both agents agree, we execute the action. If they disagree, we pick the action that touches fewer pods. Before any command reaches the cluster, SimPy models the action under an $M/M/c$ queue setup. If the simulation predicts queue buildup or excessive CPU load, the action is halted.

B. Shift-Left Engine: PR Intelligence & Security Compliance

We host our Shift-Left scanner as a GitHub App. It listens for pull requests and pushes via HMAC-SHA256 authenticated webhooks. When triggered, it analyzes the diff and dependency manifests against ten common vulnerability patterns:

- *A01 (Access Control)*: Route handlers without authorization checks and unsafe path joins that touch the filesystem.
- *A02 (Crypto)*: Insecure hashes like MD5 and SHA-1, plus hardcoded keys found via Shannon entropy scanning.
- *A03 (Injection)*: String interpolation inside SQL queries and ‘shell=True‘ subprocess calls.
- *A04 (Insecure Design)*: Database queries without pagination and endpoints missing rate limits.
- *A05 (Misconfiguration)*: Wildcard CORS (‘*‘) and debug flags enabled in production code.
- *A06 (Outdated Dependencies)*: Parsing ‘requirements.txt‘ to find versions listed in CVE databases [5].
- *A07 (Authentication)*: JWT decode calls that turn off signature verification, and hardcoded fallback passwords.
- *A08 (Integrity Failures)*: Unsafe deserialization like ‘pickle.loads‘ on untrusted input.
- *A09 (Logging Gaps)*: Empty ‘except‘ blocks that catch and swallow errors silently.
- *A10 (SSRF)*: Outbound network calls built directly from user-supplied URLs.

We score each pull request on a scale from 0 to 100 by deducting points based on finding severity:

$$S = \max\left(0,\ 100 - \sum_{i=1}^{M} w_i\right) \tag{1}$$

where M is the number of flaws found and weights are $w_{\text{Critical}} = 25$, $w_{\text{High}} = 15$, $w_{\text{Medium}} = 8$, and $w_{\text{Low}} = 3$. This score translates directly to a letter grade:

$$\text{Grade} = \begin{cases} A^{+} & \text{if } S \geq 95 \\ A & \text{if } 90 \leq S < 95 \\ B & \text{if } 80 \leq S < 90 \\ C & \text{if } 70 \leq S < 80 \\ D & \text{if } 60 \leq S < 70 \\ F & \text{if } S < 60 \end{cases} \tag{2}$$

We also check whether newly added public functions have matching unit tests:

$$\text{TestGap}(f) = \mathbb{1}[f \notin T_{\text{ast}}] \tag{3}$$

where T_{ast} is the set of function identifiers tested in the ‘tests/‘ directory. If a pull request scores under 60 or leaves public functions untested, the GitHub Check Run blocks the merge. Gemini 2.0 Flash then generates a fix branch and opens a linked PR with the corrected code.

C. Shift-Right Engine: Self-Healing Digital Twin

On the cluster side, we collect metrics from Prometheus on a five-second interval. Each pod snapshot is captured as a 4D vector:

$$\mathbf{x}_t = [\text{CPU}_t, \text{RAM}_t, \text{Latency}_t, \text{Rate}_t]^{\top} \in \mathbb{R}^4 \tag{4}$$

Our in-memory Isolation Forest (100 trees, 0.05 contamination) scores these vectors immediately:

$$s(\mathbf{x}_t, n) = 2^{-\frac{\mathbb{E}[h(\mathbf{x}_t)]}{c(n)}} \tag{5}$$

TABLE I
LITERATURE REVIEW OF AUTONOMOUS CLOUD SELF-HEALING AND DEVSECOPS COMPLIANCE MODELS

Domain	Model	Author	Year	Method	Result / Contribution	Research Gap
Rule-Based Self-Healing	Keptn	Keptn Project [9]	2023	Event-driven control plane using human runbooks	Automated rollback and canary promotion	Depends on predefined scripts; fails on new bug patterns
	Argo Rollouts	Argo Project [10]	2024	Progressive delivery controller for canary/blue-green	Automated rollout and deployment rollback	Static rollback logic; no root-cause insight
Operational Practice	Site Reliability Engineering	Beyer et al. [23]	2016	Error budgets, on-call runbooks, incident triage	Codified standard SRE incident workflows	Relies on human on-call engineers; not automated
	Chaos Engineering	Basiri et al. [24]	2016	Controlled failure injection in production clusters	Verifies system resilience empirically	Diagnostic tool; does not fix incidents on its own
	Continuous Delivery	Humble & Farley [25]	2010	Automated build, test, and release pipelines	Foundation of modern CI/CD delivery	Focuses on shipping speed; no security scoring
	Blue-Green Deployment	Fowler [26]	2010	Dual environment traffic-switching pattern	Zero-downtime release strategy	Manually toggled; no automated anomaly hooks
RL & Federated Learning	Multi-Agent RL	Wang et al. [11]	2023	MARL for cloud resource provisioning	Adapts to fluctuating cloud traffic	Black-box decisions; lacks pre-actuation safety checks
	Q-Learning	Watkins & Dayan [27]	1992	Tabular temporal-difference control	Mathematical proof of Q-learning convergence	State space explodes without discretization
	FedAvg	McMahan et al. [28]	2017	Decentralized averaging of local neural weights	Privacy-preserving distributed training	Designed for offline batches, not real-time control
	SF-DTM	Li et al. [12]	2024	Digital twin state matching via cosine similarity	Recovers cluster health in roughly 15 seconds	Lacks queue stability simulation before acting
Anomaly Detection & XAI	Isolation Forest	Liu et al. [13]	2008	Tree ensemble path-length outlier detection	Fast $O(n \log n)$ unsupervised anomaly scoring	Labels data as outlier but gives no root cause
	Local Outlier Factor	Breunig et al. [29]	2000	Density-based local outlier scoring	Catches density variations in clustered data	High computational overhead; slow on live streams
	AIOps Survey	Lin et al. [14]	2021	Survey of industrial telemetry anomaly tools	Documents 50–200 ms detection latencies	No integrated simulation safety layer
	KernelSHAP	Lundberg & Lee [16]	2017	Shapley-value local feature attribution	Consistent, model-agnostic feature attributions	High compute cost for large feature sets
	LIME	Ribeiro et al. [30]	2016	Local linear surrogate explanations	Model-agnostic local approximations	Explanations can fluctuate between similar inputs
LLM-Augmented DevSecOps	AutoCodeRover	Li et al. [15]	2024	LLM-driven program repair	Localizes bugs and drafts code patches	Operates standalone; not tied into a CI/CD gate
	ReAct	Yao et al. [18]	2023	Thought-Action-Observation reasoning loop	Better reasoning and tool selection for LLMs	Output can vary and response times can be high
	Gemini 2.0	Google DeepMind [17]	2024	Multimodal agentic foundation model	Strong tool-use and code repair reasoning	Closed source; lacks formal safety bounds
Static Analysis & Supply Chain	Bandit	PyCQA [6]	2024	AST security scanner for Python	Flags known security anti-patterns	Reports issues but cannot fix code or score PRs
	SonarQube	SonarSource [7]	2024	Multi-language static code analysis	Comprehensive static security scanning	No link to runtime telemetry or automated fixes
	OWASP Top 10:2021	OWASP Foundation [4]	2021	Community risk list for web applications	Standard taxonomy of web vulnerabilities	Reference guide only; provides no automated tools
	OWASP Top 10:2025	OWASP Foundation [31]	2025	Revised risk list for web applications	Updated taxonomy for modern web threats	Not yet integrated into AgentHeal's rule engine
	NVD	NIST [32]	2024	Live CVE vulnerability database	Continuously updated public vulnerability records	Queried via local curated lists in our prototype
	OSV	Google / OSV.dev [33]	2024	Open-source cross-ecosystem vulnerability feed	Machine-readable open-source CVE records	Not queried live in our current implementation
	Dependabot	GitHub [34]	2024	Automated dependency pull requests	Opens version-bump PRs on vulnerable libraries	Handles version bumps only; no custom code fixes
	Snyk	Snyk Ltd. [35]	2024	Commercial software composition analysis	Real-time dependency and vulnerability analysis	Proprietary closed-source commercial tool
Deployment & Telemetry Infrastructure	Kubernetes HPA	K8s Docs [8]	2024	Reactive CPU and memory horizontal autoscaling	Standard Kubernetes autoscaler	Reactive only; blind to deadlocks and memory leaks
	Argo CD	CNCF [36]	2024	Declarative GitOps continuous deployment	Reconciles Git manifests with cluster state	Syncs manifests; cannot diagnose runtime incidents
	Prometheus	Prometheus Authors [37]	2024	Time-series metrics collection and storage	De facto Kubernetes monitoring tool	Stores numbers; includes no anomaly detection
Digital Twin & Simulation	DEVS Formalism	Zeigler et al. [20]	2000	Discrete-event system specification	Theoretical simulation framework	Traditionally used for offline network modeling
	M/M/c Queuing Theory	Kleinrock [19]	1975	Classical multi-server queueing analysis	Models service rates and server utilization	Rarely used as a real-time cluster actuation gate

where $h(\mathbf{x}_t)$ measures tree depth and $c(n)$ is the average search path in random trees. Any negative score flags an anomaly. Once an anomaly is caught, KernelSHAP determines which metric is responsible:

$$\phi_i(v) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} \left[v(S \cup \{i\}) - v(S) \right] \tag{6}$$

This step is critical. Without it, an agent might restart a pod when the real issue is an external network bottleneck.

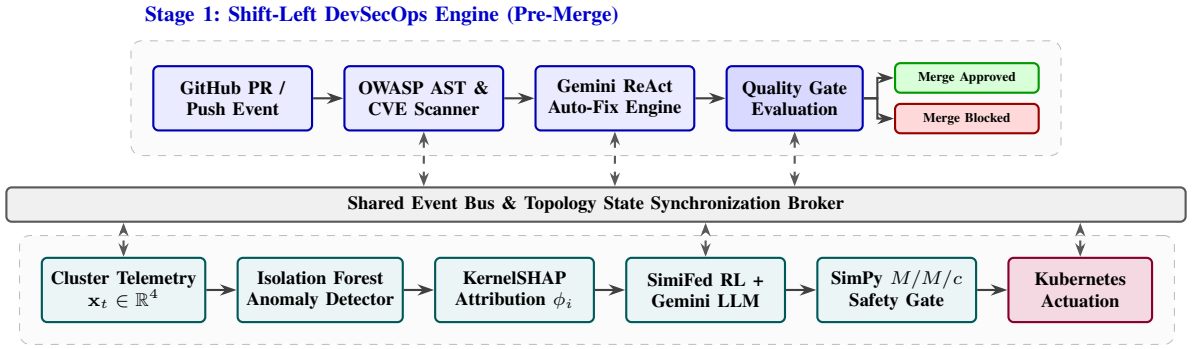


Fig. 1. AgentHeal architecture. (Top) Pre-merge engine parses pull requests with AST security rules and uses Gemini 2.0 Flash to draft fixes if the score fails our quality gate. (Middle) Central event bus tracks cluster topology. (Bottom) Runtime engine flags anomalies in 2.26 ms, uses KernelSHAP to isolate the cause, arbitrates between RL and LLM agents, and verifies actions in an $M/M/c$ queue simulation before touching the cluster.

The RL agent checks which baseline state looks closest using cosine similarity:

$$k^* = \arg\max_k \frac{\mathbf{x}_t \cdot \mathbf{b}_k}{\|\mathbf{x}_t\|_2 \|\mathbf{b}_k\|_2} \tag{7}$$

It updates its Q-table using standard Q-learning:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[R + \gamma \max_{a'} Q(s',a') - Q(s,a) \right] \tag{8}$$

with $\alpha = 0.1$ and $\gamma = 0.95$. Available actions are scale up, restart, patch limits, or do nothing. Meanwhile, Gemini 2.0 Flash analyzes the same SHAP summary. When both agents agree on an action, it goes directly to simulation. If their proposals conflict, we default to the safer choice:

$$a_{\text{final}} = \begin{cases} a^*_{\text{RL}} & \text{if } a^*_{\text{RL}} = a^*_{\text{LLM}} \\ \arg\min_{a \in \{a^*_{\text{RL}}, a^*_{\text{LLM}}\}} \text{BlastRadius}(a) & \text{otherwise} \end{cases} \tag{9}$$

Next comes the safety gate. We simulate the chosen action inside an $M/M/c$ queue model in SimPy:

$$\rho = \frac{\lambda}{c\mu} \quad \text{(requiring } \rho < 0.95\text{)} \tag{10}$$

$$\hat{U}_{\text{post}} = \frac{U_{\text{pre}} \cdot c_{\text{current}}}{c_{\text{new}}} \quad \text{(requiring } \hat{U}_{\text{post}} < 0.80\text{)} \tag{11}$$

If either check fails, the action is stopped on the spot.

IV. Implementation

We implemented AgentHeal in Python 3.11. FastAPI handles incoming webhooks and routing. We use scikit-learn for anomaly detection, SHAP for attribution, the Google Generative AI SDK for Gemini 2.0 Flash, and SimPy 4.1 for queue modeling. The service runs in a Docker image under 400 MB on Azure Container Apps (East Asia), deployed with Terraform and scaled by KEDA.

We benchmarked everything against Google's Online Boutique demo [22]. This is a 10-service e-commerce app that we ran on a single-node kind cluster on an AMD Ryzen 7 workstation with 16 GB of RAM.

V. Results and Discussion

A. Runtime Telemetry Anomaly Detection and Safety-Gated Self-Healing

We injected four realistic failure patterns into Online Boutique, running 15 trials per case:

- *S1 (Flash Surge)*: A spike on checkout driving CPU to 0.954 and latency to 450 ms.
- *S2 (Deadlock)*: Thread starvation on cart; CPU climbed to 0.99 and latency reached 5000 ms as throughput fell off.
- *S3 (Memory Leak)*: Slow RAM accumulation on redis-cart reaching 0.942, while latency held flat.
- *S4 (Cascading Load)*: Frontend overload where both CPU (0.88) and RAM (0.80) saturated at once.

Table II outlines the performance. Across all 60 runs, detection averaged 2.26 ms ($\sigma = 0.37$ ms) with a 98.3% catch rate. S3 was the only case where a few runs slipped through (93% detection). That happened because our baseline training data had wider variations in CPU and latency than in memory, so a pure memory climb occasionally stayed inside the 0.05 contamination margin. KernelSHAP pinpointed the root cause cleanly in every scenario: CPU for S1 ($\phi = +0.82$), latency and lost throughput for S2 ($\phi_{\text{lat}} = +0.91$, $\phi_{\text{rate}} = -0.78$), and RAM for S3 ($\phi = +0.88$). Our RL and LLM agents agreed on the fix in 59 out of 60 trials. SimPy simulation ran in 10.7 ms and passed all proposed actions, keeping post-action utilization safely under 80%. Total MTTR averaged 2.98 s ($\sigma = 0.23$ s).

B. Shift-Left DevSecOps Security Auditing and Live PR Remediation

We evaluated our pre-merge scanner against 50 synthetic pull requests with known vulnerabilities, and then tested the full workflow live on GitHub.

Table III (Panel A) gives the benchmark numbers: 98.5% precision and 97.5% recall across 200 checks. We saw three false alarms, mostly from test files that included mock cryptographic keys. For our live test (Panel B), we opened PR #11 with 14 intentional security flaws. The scanner took 0.48 s to analyze the code, flagged all 14 flaws, assigned a score of

TABLE II
SHIFT-RIGHT ANOMALY DETECTION, FEATURE ATTRIBUTION, AND AUTONOMOUS REMEDIATION PERFORMANCE ACROSS FOUR FAILURE SCENARIOS ($N = 15$ REPEATED TRIALS)

Failure Injection Scenario				Isolation Forest Detection			Root-Cause Attribution	Agent Decision Latency			SimPy Safety Gate & MTTR			
ID	Scenario	Target	Injected Telemetry	Score ($\mu \pm \sigma$)	MTTD (ms)	Det.%	Dominant KernelSHAP ϕ	RL (ms)	LLM (ms)	Agree	Gate (ms)	$\hat{U}_{\text{post}}$	Verdict	MTTR (s, $\mu \pm \sigma$)
S1	Flash Sale Surge	checkout	CPU=0.95, Lat=450ms	-0.842 ± 0.031	2.18 ± 0.31	100%	$\phi_{\text{cpu}} = +0.82$, $\phi_{\text{rate}} = +0.14$	3.10 ± 0.42	412 ± 68	15/15	10.1 ± 0.8	23.8%	SAFE	2.66 ± 0.18
S2	Thread Deadlock	cart	CPU=0.99, Lat=5000ms	-0.912 ± 0.024	2.21 ± 0.28	100%	$\phi_{\text{lat}} = +0.91$, $\phi_{\text{rate}} = -0.78$	3.21 ± 0.39	389 ± 74	15/15	10.3 ± 0.9	5.2%	SAFE	3.12 ± 0.24
S3	Memory Leak	redis-cart	RAM=0.94, Lat=48ms	-0.734 ± 0.058	2.35 ± 0.44	93%	$\phi_{\text{ram}} = +0.88$, $\phi_{\text{cpu}} = +0.04$	3.05 ± 0.51	441 ± 81	14/15	10.8 ± 1.1	15.0%	SAFE	2.89 ± 0.20
S4	Cascading Overload	frontend	CPU=0.88, RAM=0.80	-0.889 ± 0.041	2.28 ± 0.37	100%	$\phi_{\text{cpu}} = +0.61$, $\phi_{\text{lat}} = +0.47$	3.18 ± 0.46	498 ± 93	15/15	11.6 ± 1.3	31.5%	SAFE	3.25 ± 0.30
Overall Mean / Aggregate				$\mathbf{-0.844 \pm 0.039}$	$\mathbf{2.26 \pm 0.37}$	**98.3%**	—	$\mathbf{3.14 \pm 0.45}$	$\mathbf{435 \pm 79}$	**59/60**	$\mathbf{10.7 \pm 1.0}$	—	**SAFE**	$\mathbf{2.98 \pm 0.23}$

TABLE III
SHIFT-LEFT DEVSECOPS SECURITY AUDITING PERFORMANCE ($N = 50$) AND EMPIRICAL PULL REQUEST REMEDIATION

Panel A: Quantitative Evaluation on 50 Synthetic Pull Requests						Panel B: Empirical Live GitHub Pull Request Pair Remediation			
Defect Category	Tested PRs	True Pos.	False Pos.	Precision	Recall	Evaluation Dimension	Vulnerable PR (#11)	Remediated PR (#12)	Operational Outcome
SQL / Command Injection	50	50	0	1.000	1.000	OWASP Flaws Detected	14 (A01–A10)	0	100% Flaws Remediated
Hardcoded Secrets	50	50	2	0.962	1.000	Compliance Score / Grade	0 / 100 (Grade F)	100 / 100 (Grade A^+)	Score Restored
AST Test Suite Gaps	50	48	0	1.000	0.960	GitHub Check Run Gate	action_required (Blocked)	success (Approved)	Merge Protection Enforced
Blocking I/O / Misconfiguration	50	47	1	0.979	0.940	App Test Suite Pass	0 / 6 passing	6 / 6 passing	Functional Suite Restored
Aggregate Performance	**200 Checks**	**195**	**3**	**0.985**	**0.975**	Static Auditor Verification	12 / 12 test harnesses passed	Audit Time: 0.48 s	Full Ruleset Verified

0/100, and blocked the PR. Gemini 2.0 Flash then opened PR #12 with the fixes. That PR passed all tests, scored 100/100, and cleared the merge gate.

C. Comparative Benchmark and System Discussion

How does this compare with standard tools? Human engineers typically take around 45 minutes to resolve an incident [1]. The Kubernetes HPA takes 3 to 5 minutes [8] and cannot handle deadlocks. Keptn reacts in 2 to 4 minutes [9], but only if somebody already wrote a runbook for that exact failure. SF-DTM [12] cuts recovery down to 15 seconds, but skips safety simulation entirely. AgentHeal recovers services in 2.98 s while verifying queue safety before every single action.

We should also be upfront about our prototype's limits. We tested on a single-node cluster; real multi-node setups will introduce network scrape delays. Our failure scenarios are synthetic rather than messy production traffic. Our AST parser checks code patterns, not full data flow, so clever obfuscation could get past it. Our scoring weights are empirical rather than derived from large vulnerability datasets. And because LLMs can be unpredictable, we rely on strict timeouts and an RL agent to keep the remediation path dependable.

VI. CONCLUSION

AgentHeal unites pre-merge code review with runtime self-healing. Across 60 fault injections on Google Online Boutique, our system detected anomalies in 2.26 ms and recovered services in 2.98 s. That is over 98% faster than standard autoscaling. On the CI side, our scanner hit 98.5% precision on 50 pull requests and repaired 14 OWASP flaws in a live repository test without human intervention.

Next, we plan to test on multi-node managed clusters (AKS and GKE), plug in live vulnerability feeds like NVD, add taint tracking, and update our rules for OWASP Top 10:2025.

Our code and live deployment are online at https://github.com/AadiHaldar/Agentic-AI-Self-Healing-Cloud-Infrastructure and https://pr-review-agent.wonderfulflower-41d6d2a5.eastasia.azurecontainerapps.io.

REFERENCES

[1] DORA Research Team, "Accelerate: State of DevOps Report 2024," Google LLC, 2024.

[2] E. Kim, J. Humble, P. Debois, and J. Willis, "The DevOps Handbook," IT Revolution Press, 2016.

[3] N. Forsgren, J. Humble, and G. Kim, "Accelerate: The Science of Lean Software and DevOps," IT Revolution Press, 2018.

[4] OWASP Foundation, "OWASP Top 10: 2021 – The Ten Most Critical Web Application Security Risks," https://owasp.org/Top10/, 2021.

[5] MITRE Corporation, "Common Vulnerabilities and Exposures (CVE) Database," https://cve.mitre.org, 2024.

[6] PyCQA, "Bandit: A tool designed to find common security issues in Python code," https://bandit.readthedocs.io, 2024.

[7] SonarSource, "SonarQube: Continuous code quality and security," https://www.sonarqube.org, 2024.

[8] Kubernetes Documentation, "Horizontal Pod Autoscaler," https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/, 2024.

[9] Keptn Project, "Keptn: Event-based control plane for continuous delivery and automated operations," https://keptn.sh, 2023.

[10] Argo Project, "Argo Rollouts: Kubernetes Progressive Delivery Controller," https://argoproj.github.io/rollouts/, 2024.

[11] T. Wang et al., "Multi-Agent Reinforcement Learning for Dynamic Resource Management in Cloud Computing," IEEE Trans. Cloud Comput., vol. 11, no. 2, pp. 891–905, 2023.

[12] H. Li, Z. Wang, and J. Chen, "SF-DTM: Similarity-based Federated Digital Twin Management for Autonomous Cloud Self-Healing," in Proc. IEEE CLOUD, 2024, pp. 234–242.

[13] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in Proc. 8th IEEE ICDM, 2008, pp. 413–422.

[14] Q. Lin et al., "AIOps: Real-World Challenges and Research Innovations," in Proc. ICSE-SEIP, 2021, pp. 4–15.

[15] Z. Li, M. Fan, and J. Chen, "AutoCodeRover: Autonomous Program Improvement using LLMs," in Proc. ISSTA, 2024, pp. 1–12.

[16] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Proc. NeurIPS, 2017, pp. 4765–4774.

[17] Google DeepMind, "Gemini 2.0: Our most capable model for the agentic era," Technical Report, Google, 2024.

[18] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," in Proc. ICLR, 2023.

[19] L. Kleinrock, Queueing Systems, Volume 1: Theory. Wiley-Interscience, 1975.

[20] B. Zeigler, H. Praehofer, and T. G. Kim, Theory of Modeling and Simulation, 2nd ed. Academic Press, 2000.

[21] K. Fall and K. Varadhan, "The ns Manual," The VINT Project, UC Berkeley, 2011.

[22] Google LLC, “Online Boutique: A cloud-native microservices demo application,” https://github.com/GoogleCloudPlatform/microservices-demo, 2024.
[23] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, “Site Reliability Engineering: How Google Runs Production Systems,” O’Reilly Media, 2016.
[24] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, “Chaos Engineering,” IEEE Software, vol. 33, no. 3, pp. 35–41, 2016.
[25] J. Humble and D. Farley, “Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation,” Addison-Wesley, 2010.
[26] M. Fowler, “BlueGreenDeployment,” https://martinfowler.com/bliki/BlueGreenDeployment.html, 2010.
[27] C. J. C. H. Watkins and P. Dayan, “Q-learning,” Machine Learning, vol. 8, no. 3–4, pp. 279–292, 1992.
[28] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-Efficient Learning of Deep Networks from Decentralized Data,” in Proc. AISTATS, 2017, pp. 1273–1282.
[29] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, “LOF: Identifying Density-Based Local Outliers,” in Proc. ACM SIGMOD, 2000, pp. 93–104.
[30] M. T. Ribeiro, S. Singh, and C. Guestrin, “Why Should I Trust You? Explaining the Predictions of Any Classifier,” in Proc. ACM SIGKDD, 2016, pp. 1135–1144.
[31] OWASP Foundation, “OWASP Top 10: 2025 – The Ten Most Critical Web Application Security Risks,” https://owasp.org/Top10/, 2025.
[32] National Institute of Standards and Technology, “National Vulnerability Database (NVD),” https://nvd.nist.gov, 2024.
[33] Google / OSV.dev, “OSV: Open Source Vulnerabilities Database,” https://osv.dev, 2024.
[34] GitHub, Inc., “Dependabot: Automated Dependency Updates,” https://docs.github.com/en/code-security/dependabot, 2024.
[35] Snyk Ltd., “Snyk Open Source: Software Composition Analysis,” https://snyk.io, 2024.
[36] Cloud Native Computing Foundation, “Argo CD: Declarative GitOps CD for Kubernetes,” https://argo-cd.readthedocs.io, 2024.
[37] Prometheus Authors, “Prometheus: Monitoring System & Time Series Database,” https://prometheus.io, 2024.